

CS246 Spring 2026 Project – CC3k

C. Kierstead, A. Reetz

- DO NOT EVER SUBMIT TO MARMOSET WITHOUT COMPILING AND TESTING FIRST. If your final submission doesn't compile, or otherwise doesn't work, you will have nothing to show during your demo. Resist the temptation to make last-minute changes. They probably aren't worth it.
- This project is intended to be doable by three people in two weeks. Because the breadth of students' abilities in this course is quite wide, exactly what constitutes two weeks' worth of work for three students is difficult to nail down. Some groups will finish quickly; others won't finish at all. We will attempt to grade this assignment in a way that addresses both ends of the spectrum. You should be able to pass the assignment with only a modest portion of your program working. Then, if you want a higher mark, it will take more than a proportionally higher effort to achieve, the higher you go. A perfect score will require a complete implementation. If you finish the entire program early, you can add extra features for a few extra marks.
- Above all, MAKE SURE YOUR SUBMITTED PROGRAM RUNS. The markers do not have time to examine source code and give partial correctness marks for non-working programs. So, no matter what, even if your program doesn't work properly, make sure it at least does something.

1 The Game of ChamberCrawler3000

In this project, you and your group will work together to produce the video game ChamberCrawler3000 (CC3k), which is a simplified rogue-like (a genre of video game based upon the game Rogue¹ - [https://en.wikipedia.org/wiki/Rogue_\(video_game\)](https://en.wikipedia.org/wiki/Rogue_(video_game))).

A game of CC3k consists of a board 79 columns wide and 30 rows high (5 rows are reserved for displaying information). Game play is as follows: the player character moves through a dungeon and slays enemies and collects treasure until reaching the end of the dungeon (where the end of the dungeon is the 5th floor). A dungeon consists of different floors that consist of chambers connected with passages. In our simplification, each floor will always consist of the same 5 chambers connected in exactly the same way (outlined below).

CC3k differs from other rogue-likes in a significant way: it does not update the terminal/window in real-time but rather redraws all elements every turn (e.g. after every command). Many early rogue-likes were programmed in a similar fashion to CC3k.

However, strange happenings are afoot in the chamber this term. The monsters have revolted and demanded to be made the heroes due to their negative portrayal in previous iterations of CC3k! Accordingly, we have acquiesced² and so we have a new group of heroic adventurers this term.

1.1 Some Definitions

It is understandable that this type of game may be new to readers of this document³. Accordingly, some definitions are provided here to aid in the reading of this document.

Definition 1: A **character** is a person/animal/thing in the game of CC3k. This can be either the player character (PC), who is controlled by the player of the game, or non-playable characters, who are strictly enemies in CC3k.

Definition 2: An **item** is something the player character can pick up or use. In CC3k, this is either gold or potions. Potions offer potentially positive and negative effects to the player character.

Definition 3: A **chamber** is an individual room in the game of CC3k. Chambers are connected by **passages**.

¹Which is itself based on Dungeons and Dragons.

²They're monsters after all.

³In the internet vernacular, y'all be n00bz.

Definition 4: A **floor** in CC3k is a predefined configuration of 5 chambers with connecting passageways. Figure 1 depicts an empty floor. Note that the configuration is the same for every floor in a game of CC3k.

Definition 5: **Health Points (HP)** is the representation of a character's health (both enemies and the player character). When a character's HP reaches 0, they are slain. For an enemy this means that they are removed from the floor and a tidy sum of gold is given to the player character. When the player character has 0 HP then the current game ends.

Definition 6: **Attack (Atk)** is the representation of a character's strength. This is how hard a character can hit another character. In CC3k conflict is solely between the player character and non-playable characters.

Definition 7: **Defense (Def)** is the representation of a character's toughness. This is how hard a character can be hit by another character.

Definition 8: A **cell** is either a wall, floor tile, doorway, or passage.

Definition 9: Something being **spawned** means that the particular something (an enemy, gold, etc) should be generated and placed on the board.

Definition 10: A **1 block radius** denotes the 8 cells adjacent to the character or item. If the character is near an edge or a corner, there are fewer adjacent cells.

2 System Components

The major components of the system are as follows:

2.1 Player Character

The default player character race is a shade⁴ that has starting stats (125 HP, 25 Atk, 25 Def). However, players have the option of choosing an alternate (but no less heroic) race: drow (150 HP, 25 Atk, 15 Def, all potions have their effect magnified by 1.5), vampire (50 HP, 25 Atk, 25 Def, gains 5 HP every successful attack and has no maximum HP), troll (120 HP, 25 Atk, 15 Def, regains 5 HP every turn; HP is capped at 120 HP), and goblin (110 HP, 15 Atk, 20 Def, steals 5 gold from every slain enemy). Max HP for all races is the starting HP, except for vampires that have no maximum.

In our game board, the player character is always denoted by the '@' symbol.

Question. How could you design your system so that each race could be easily generated? Additionally, how difficult does such a solution make adding additional races?

2.2 Enemies

Enemies are the mortal foes of our illustrious player character. In a traditional rogue-like game, the enemy character would have some degree of artificial intelligence. However, for simplicity in CC3k, enemies, except for dragons, move one square randomly within the confines of the chamber they were spawned in. Dragons are stationary and always guard a treasure hoard.

Enemies can be one of human (140 HP, 20 Atk, 20 Def, drops 2 normal piles of gold), dwarf (100 HP, 20 Atk, 30 Def, Vampires are allergic to dwarves and lose 5 HP rather than gain), elf (140 HP, 30 Atk, 10 Def, gets two attacks against every race except drow), orcs (180 HP, 30 Atk, 25 Def, does 50% more damage to goblins), merchant (30 HP, 70 Atk, 5 Def), dragon (150 HP, 20 Atk, 20 Def, always guards a treasure hoard), and halfling (100 HP, 15 Atk, 20 Def, has a 50% chance to cause the player character to miss in combat, i.e. takes priority over player character's ability to never miss).

By default, merchants are neutral to all parties⁵. However, merchants can be attacked and slain by the player character. Attacking or slaying a Merchant causes every Merchant from that point forward to become hostile to the player character (and will attack them if they pass within a one block radius).

Dragons always spawn in a one block radius of its dragon hoard (see Treasure). That is, if a dragon hoard is spawned then a dragon is spawned.

⁴A human who has become aligned with dark forces. Possibly CS 246 Course Staff.

⁵They're capitalists after all

Upon their demise, any enemy that is not a dragon, human, or merchant will drop either a small pile or normal pile of gold (discussed below). This gold is immediately added to the player character's total.

Enemies (except dragons, who are stationary) move randomly 1 floor tile at a time, assuming the floor tile is unoccupied (see Section 3 for floor tile description). An enemy can never leave the room it was spawned (created) in. Note that enemies should be moved in a line-by-line fashion. That is, starting at the leftmost enemy, move all enemies on that row and then move to the next row starting with the leftmost. Any particular enemy should only be moved once per player action (e.g. moving to a line that has not been processed does not grant an extra move). However, should the player character be within a 1 block radius of an enemy then the enemy will always attack the player character.

Enemies are denoted on the map as follows: (H)uman, d(W)arf, (E)lf, (O)rc, (M)erchant, (D)ragon, Half(L)ing.

Question. How does your system handle generating different enemies? Is it different from how you generate the player character? Why or why not?

Question. How could you implement the various abilities for the enemy characters? Do you use the same techniques as for the player character races? Explain.

2.3 Items

2.3.1 Potions

In the game of CC3k, there is only one type of usable item: a potion. Potions are of two types: positive and negative. Potions can provide the player character with positive and negative bonuses as outlined below. Regardless of the potion itself, all potions are denoted on the map with a P. A player may not hold any potions⁶. Accordingly, a potion cannot be used unless a player is standing within a 1 block radius of it.

The effects of a particular potion are not known until it is used for the first time, e.g. the player character will not know what a potion does until they use it for the first time in a session. However, they will only learn about the effects of that particular potion. Other potions will not have their effects revealed. The exception is for potions purchased from the Merchant⁷, which should have have their abilities displayed.

Positive Potions:

- Restore health (RH): restore up to 10 HP (cannot exceed maximum prescribed by race)
- Boost Atk (BA): increase ATK by 5
- Boost Def (BD): increase Def by 5

Negative Potions:

- Poison health (PH): lose up to 10 HP (cannot fall below 0 HP)
- Wound Atk (WA): decrease Atk by 5
- Wound Def (WD): decrease Def by 5

The effects of RH and PH are permanent while the effects of all other potions are limited to the floor they are used on. For example, using a BA potion will only boost the player character's Atk until the beginning of the next floor.

Note that the PC's Atk and Def can never drop below 0.

Question. What design pattern could you use to model the effects of temporary potions (Wound/Boost Atk/Def) so that you do not need to explicitly track which potions the player character has consumed on any particular floor?

⁶Note: Adding an inventory is a great DLC for CC3k.

⁷Only a concern for bonus purposes. The default game does not require Merchants to sell anything.

2.3.2 Treasure

Treasure in CC3k consists only of gold. Gold can be in several types of piles: small (value 1), normal (value 2), merchant hoard (value 4), and dragon hoard (value 6). Recall, a dragon must always protect a dragon hoard whenever it randomly spawns. A dragon hoard can only be picked up once the dragon guarding it has been slain. Gold, regardless of type, is denoted by 'G' on the map.

A merchant hoard is dropped upon the death of a merchant. Gold dropped by a merchant (merchant hoard, value 4) or a human (2 normal piles, value $2 \times 2 = 4$) is picked up only when the PC walks over it.

Question. How could you generate items so that the generation of Treasure and Potions reuses as much code as possible? That is, how would you structure your system so that the generation of a potion and then generation of treasure does not duplicate code?

2.4 Floors

Levels are generated to consist of the 5 chambers connected in the manner outlined in Figure 1. It would be more interesting to have randomly connected randomly generated chambers but that is more complicated than the time frame allows⁸.

The player character should spawn randomly in a chamber (every chamber is equally likely) but it should never be the case that the player spawns in the chamber with the stairs going down to the next level. Stairs are denoted by '\'. Note that the stairway and player character may be spawned with equal probability on any floor tile in a chamber. That is, a larger chamber should be no more likely to spawn the PC/stairs than a smaller chamber, where any floor tile in the selected chamber is equally likely to spawn the PC/stairs.

10 potions are spawned on each floor. A potion's type is chosen at random, with each type having equal (1/6) probability. The chamber the potion spawns in is also chosen at random, with each room having equal (1/5) probability. Each square in the chamber a potion spawns in has an equal chance of containing the potion. Note that this means in particular that small rooms are just as likely to spawn potions as large rooms.

We might like to have gold spawn more or less frequently as the game gets more difficult. However, to again simplify design the spawn rate of gold is 5/8 chance of normal, 1/8 dragon hoard, 1/4 small hoard. Chambers are equally likely (as are floor tiles in any particular chamber) to spawn gold. 10 piles of gold are spawned on every floor.

With the exception of dragons, enemies have the following probability distribution of being spawned:

- Human: 2/9
- Dwarf: 3/18
- Halfling: 5/18
- Elf: 1/9
- Orc: 1/9
- Merchant: 1/9

20 enemies are spawned per floor (this number does not include dragons). Every chamber is equally likely to spawn any particular monster (similarly for floor tiles).

We require that generation happens in the following order: player character location, stairway location, potions, gold, enemies. This is to allow us to more easily evaluate that your random generation is correctly implemented.

Note that multiple objects (enemies, gold, and potions) cannot occupy the same cell on the game board. That is, no two objects can ever occupy the same space. The one exception to this is the case of gold. Typically, when a player character walks over gold, it is picked up. The exception to this is if the gold is associated with a still alive dragon; in this case, the player simply walks over the gold, without picking it.

When the PC attempts to move on to a stairway, the next level is instead generated and displayed, with the PC spawning in a random position on the new level.

Items and enemies should only ever spawn on a floor tile and never in a doorway, passage, or the stairs leading down to the next floor.

⁸This would be a *hard* but good DLC for CC3k.

2.5 Combat

By default, all enemies except for Merchants and Dragons are hostile to the player character. If the player character enters within a 1 block radius of any hostile enemy, they will attempt to attack the player character (even before the player character has had a chance to attack). Dragons are considered hostile when the player is next to (read: in the 1 block radius of) its dragon hoard or itself, and will use their fire breath to defend its hoard (i.e. will attack the enemy). This means that a Dragon might attack even if the player is not next to the Dragon, but because it is next to a dragon hoard. If the player character is not within a 1 block radius of the enemy then it will resume random movement (as previously described). Recall that Merchants can become hostile when one is attacked/slain by the player character.

Combat is resolved as follows: Enemies will auto-attack players given the previously specified criteria, however, there is a 50% chance their attack misses. The player character has the option of attacking any of the 8 squares adjacent to them and within a 1 block radius. The PC never misses. Recall, that the PC has initiative and always attacks first.

Damage is calculated as follows: $Damage(Defender) = \text{ceiling}((100/(100 + Def(Defender))) * Atk(Attacker))$, where Attacker specifies the attacking character (enemy or PC) and defender specifies the character being attacked. Thus, in a single round a character can be both an attacker and a defender.

3 Display

The display of CC3k is relatively simple, Figure 1 depicts an empty board. Walls are denoted by ‘|’ and ‘-’, doorways by ‘+’, and passages by ‘#’. Floor tiles that can be walked upon are denoted by ‘.’. Chambers are denoted by the smaller polygons inside the larger rectangle. The player character can only ever occupy a passage block, doorway block, or a floor tile inside a chamber. The player character can see in all chambers simultaneously, e.g. through walls or doors⁹. Figures 2, 3, 4, and 5 depict various board states. Note that Figure 1 represents a completely empty game board and is meant to act as a reference of what the game board would look like before any generation occurs.

We require that you use ASCII colour output codes when outputting the board to your screen so that both you and the marker can more quickly pick out the important information.

- Display the PC and staircase in blue, so they are easy to spot. Enemies are displayed in red, treasure is displayed in yellow, and potions are displayed in green.
 - If one of your group members is colour-blind, please pick a reasonable contrasting colour that works for them.
- See <https://medium.com/@vitorcosta.matiass/print-coloured-texts-in-console-a0db6f589138> for an article on how to do this.

4 Command Interpreter

Initially, the game will demand the player enter one of the specified races or quit. Entering ‘q’ or EOF (e.g. Ctrl-D) at the race prompt will cause the program to terminate. Supplying a valid race selection (below) will start that game using that race. Other values will be ignored.

Play will continue in the specified way until the player restarts, reaches the end of floor 5, the PC dies, or the player quits. If the player reaches the end of the game or their character is slain, the game should give them the option of playing again or quitting.

The following commands can be supplied to your command interpreter:

- **no,so,ea,we,ne,nw,se,sw**: moves the player character one block in the appropriate cardinal direction.
- **u direction**: uses the potion indicated by the *direction* (e.g. no, so, ea).
- **a direction**: attacks the enemy in the specified *direction*, if the monster is in the immediately specified block (e.g. must be one block north of the @).
- **s, d, v, g, t**: specifies the race the player wishes to be when starting a game.

⁹For the sake of simplicity, the player character has X-Ray vision.

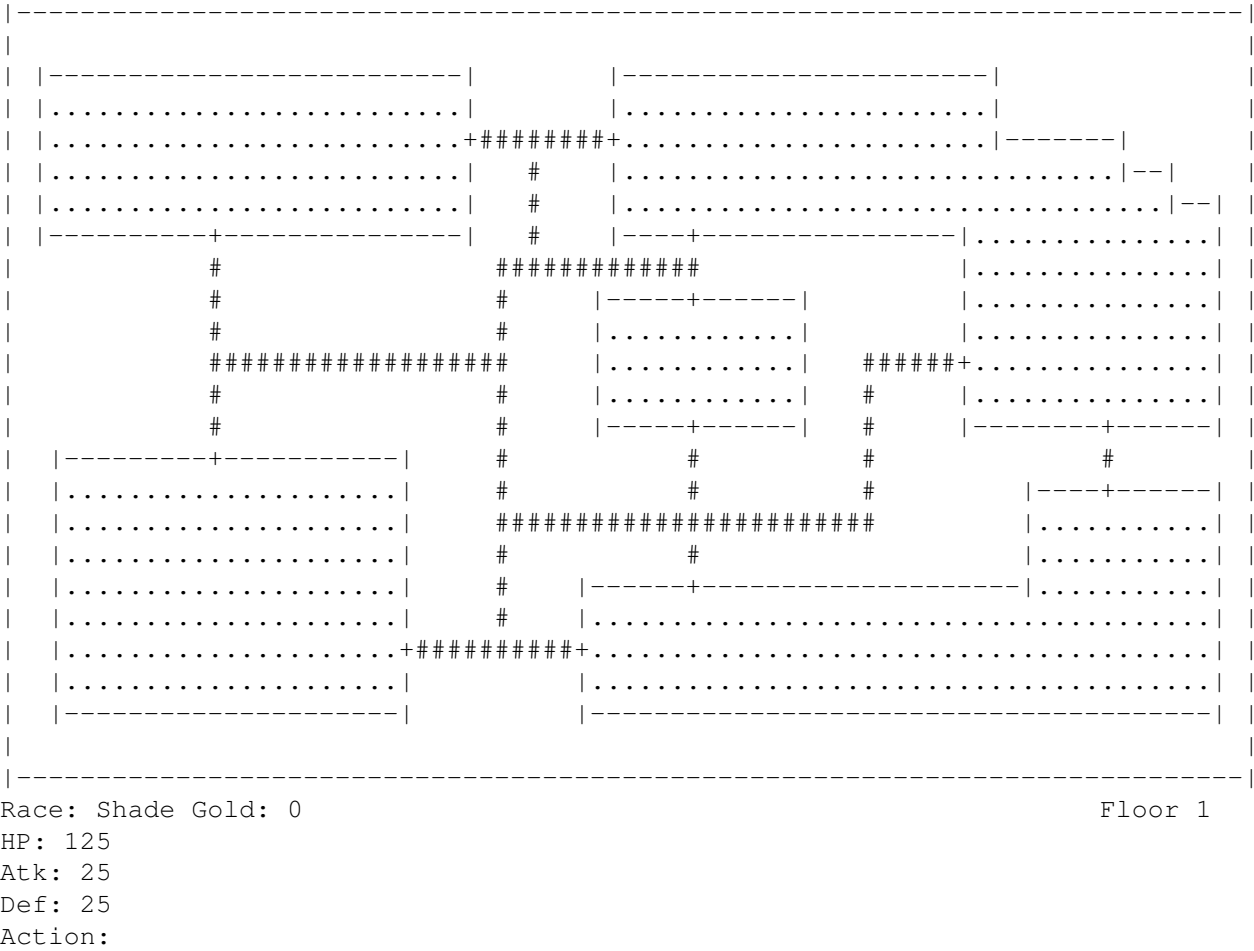


Figure 1: Empty board showing all passages between chambers.

```

|-----|
| |-----| |-----|
| |.....@.....W.....| |..L.....L.....G..|
| |...H.....E.....+#####+.....|-----|
| |.....L.....| # |.....O.....W.....|--|
| |...O.....M....| # |..P.....|-----|--|
| |-----+-----| # |-----+-----|.....|
| | # |#####| |.....G.....|
| | # | # |-----+-----| |..H.....D.....|
| | # | # |.....E...G..| |.....|
| |#####| |.....\.....| #####+...P.....|
| | # | # |.....E...P..| # |.....|
| | # | # |-----+-----| # |-----+-----|
| |-----+-----| # | # | # | # |
| |.....| # | # | # |-----+-----|
| |...O.....E.....| #####| |.....|
| |.....G.....| # | # | |..G..G...|
| |...L.....| # |-----+-----| |.....|
| |.....L.....| # |.....|
| |.....P.....+#####+...W.....P...O.....L.....|
| |..M.G.....| |.....H.....|
| |-----| |-----|
|-----|

```

Race: Shade Gold: 0 Floor 1

HP: 125

Atk: 25

Def: 25

Action: Player character has spawned.

Figure 2: A board showing a (potentially) generated board.

```

|-----|
| |-----| |-----|
| |.....@.....W.....| |..L.....L.....G..|
| |.....H.....E.....+#####+.....|-----|
| |.....L.....| # |.....O.....W.....|--|
| |.....O.....M.....| # |..P.....|-----|
| |-----+-----| # |-----+-----|.....|
| | # |#####|.....G.....|
| | # | # |-----+-----| |..H.....D.....|
| | # | # |.....E...G..| |.....|
| |#####| |.....\.....|#####+...P.....|
| | # | # |.....E...P..| # |.....|
| | # | # |-----+-----| # |-----+-----|
| |-----+-----| # | # | # | # |
| |.....| # | # | # |-----+-----|
| |...O.....E.....|#####|.....|
| |.....G.....| # | # |.....G..G.....|
| |...L.....| # |-----+-----|.....|
| |.....L.....| # |.....|
| |.....P.....+#####+...W.....P...O.....L.....|
| |...M.G.....| |.....H.....|
| |-----| |-----|
|-----|

```

Race: Shade Gold: 0

Floor 1

HP: 120

Atk: 25

Def: 25

Action: PC deals 5 damage to H (135 HP). H deals 5 damage to PC.

Figure 3: A board showing combat. (Note: Action values are fictitious and do not represent actual attack/damage calculations.)


```

|-----|
| |-----| |-----|
| |.....@P.....W.....| |..L.....L.....G..|
| |.....E.....+#####+.....|-----|
| |.....L.....| # |.....O.....W.....|--|
| |.....O.....M.....| # |..P.....|-----|--|
| |-----+-----| # |-----+-----|.....|
| | # |#####|.....G.....|
| | # | # |-----+-----| |..H.....D.....|
| | # | # |.....E...G..| |.....|
| |#####| |.....\.....|#####+.....|
| | # | # |.....E...P..| # |.....|
| | # | # |-----+-----| # |-----+-----|
| |-----+-----| # | # | # | # |
| |.....| # | # | # |-----+-----|
| |.....O.....E.....|#####|.....|
| |.....G.....| # | # |.....G..G.....|
| |.....L.....| # |-----+-----|.....|
| |.....L.....| # |.....|
| |.....P.....+#####+.....W.....P.....O.....L.....|
| |..M.G.....| |.....H.....|
| |-----| |-----|
|-----|

```

Race: Shade Gold: 1

Floor 1

HP: 100

Atk: 25

Def: 25

Action: PC moves East and sees an unknown potion.

Figure 4: A board showing that potions are unknown until used.

```

|-----|
| |-----| |-----|
| |.....@.....W.....| |..L.....L.....G..|
| |.....E.....+#####+.....|-----|
| |.....L.....| # |.....O.....W.....|--|
| |.....O.....M.....| # |..P.....|-----|
| |-----+-----| # |-----+-----|.....|
| | # |#####|.....G.....|
| | # | # |-----+-----| |..H.....D.....|
| | # | # |.....E...G..| |.....|
| |#####| |.....\.....|#####+.....|
| | # | # |.....E...P..| # |.....|
| | # | # |-----+-----| # |-----+-----|
| |-----+-----| # | # | # | # |
| |.....| # | # | # |-----+-----|
| |.....O.....E.....|#####|.....|
| |.....G.....| # | # |.....G..G.....|
| |.....L.....| # |-----+-----|.....|
| |.....L.....| # |.....|
| |.....P.....+#####+.....W.....P.....O.....L.....|
| |..M.G.....| |.....H.....|
| |-----| |-----|
|-----|

```

Race: Shade Gold: 1 Floor 1
HP: 10
Atk: 30
Def: 25
Action: PC uses BA.

Figure 5: A board showing potion usage.

- **f**: stops enemies from moving until this key is pressed again.
- **r**: restarts the game. All stats, inventory, and gold are reset. A new race should be selected.
- **q**: allows the player to admit defeat and exit the game.

Note that the board should be redrawn as appropriate every time a command is entered.

5 Ending the Game and Scoring

A game session ends in one of the following ways: the player character reaches the stairs on floor 5, the player character's health reaches 0, the player restarts the game or quits.

A player's score is only generated in the first of above three cases. Score is determined by the amount of gold they have collected in their current character's life time (except for shades who have a 50% increase to their score).

6 Command Line options

Your program **must** have the ability to process an optional single command line argument: the name of a file that specifies the layout of each of the 5 floors in order to allow the marker to test your program. This is specified by giving the exact floor layout as is presented to the player for each of the 5 floors. Essentially, any diagram presented in this document (minus the text information in the last 5 rows).

If a particular floor layout has any information other than walls/floors/halls specified, the information must be reproduced as if the particular item was generated at that location. **(It is vital that this be implemented for at least the first floor for test purposes.)** Note that potions and piles of gold will be denoted by numbers (outlined below) but should be converted to the appropriate character when displayed to the player.

The translation of numbers to items is as follows: 0 - RH, 1 - BA, 2 - BD, 3 - PH, 4 - WA, 5 - WD, 6 - normal gold pile, 7 - small hoard, 8 - merchant hoard, 9 - dragon hoard.

You may find it useful to have a second optional argument that represents a seed for your random number generation (so that you can have reproducible results). This is **not** required.

7 Grading

Your project will be graded as described in the project guidelines document.

Even if your program doesn't work at all, you can still earn a lot of marks through good documentation and design, (in the latter case, there needs to be enough code present to make a reasonable assessment).

Above all, make sure your submitted program runs, and does something! You don't get correctness marks for something you can't show, but if your program at least does something that looks like the beginnings of the game, there may be some marks available for that.

8 When all else fails...

This is a relatively complex project with many components and large amounts of random generation. Accordingly, if you find yourself running out of time or having trouble here's our suggestion for priorities:

- Reading in and printing a floor that has monsters/items/gold/exit at specified location. This is a *vital* requirement for at least the first floor since it's very difficult to mark your project without this capability.
 - Implementing this will make your demo marking go much faster since you can then set up specific test cases to demonstrate your program's functionality.
- Create general purpose player character, enemies, and items (e.g. no special types).
- Get movement and interaction working (combat, item use, etc) for these generalized versions.

- Introduce the different races and enemies.
- Introduce the different types of items.
- Random generation of items/enemies on floors.

Accomplishing the first three points should reward you with at least 50% (assuming you have used object-oriented principles to get that far). That is, you will get a higher mark for submitting something that runs but not does implement all the requirements than submitting something that attempts to implement all the requirements but doesn't run.

9 Downloadable Content (Bonus Enhancements)

The bonus enhancements for the CC3k project will be phrased in terms of free¹⁰ Downloadable Content (DLC).

You should provide some way of playing the game with and without your DLC that doesn't require recompilation (i.e. adding a new command-line parameter, or an in-game command to toggle it on/off). This will allow project assessors to accurately determine that you met all the base requirements.

However, don't attempt to implement this until you've got the base game working.

To earn significant credit, enhancements must be algorithmically difficult, or solve an interesting problem in object-oriented design. Trivial enhancements will not earn a significant fraction of the available marks.

Due Dates and Deliverables

See the project guidelines for due dates and submission requirements.

10 A Note on Random Generation

To complete this project, you will require the random generation (or rather, pseudo-random) of numbers. Games that are started with the same seed value have the same sequences of random numbers generated. There are several ways of doing this that can be seen in the provided `RandomNumberGeneratorExamples` folder.

¹⁰Contrary to the general trend in the gaming industry.